

Table 1: Services summary table

Service	Purpose	Docker image / file	Host port	Container port	Volume	User	Network	Notes
DVWA	Web app for testing vulnerabilities	<i>vulnerables/web-dvwa</i>	8080	80	N/A	Default	pentest-network	Accessible at <i>http://localhost:8080</i> , depends on <i>dvwa_db</i>
Kali Linux	GUI-enabled attack platform with tools	Custom (<i>Dockerfile.kali</i>)	7002	6901 (VNC)	<i>kali_data:/root</i>	root	pentest-network	VNC access at <i>http://localhost:7002</i> , includes essential tools
Metasploitable	Vulnerable OS for exploit practice	Custom (<i>Dockerfile.metasploitable</i>)	7003	80	<i>metasploitable_data</i>	Default	pentest-network	Legacy OS simulation accessible at <i>http://localhost:7003</i>
Snort3	Intrusion detection system (IDS)	Custom (<i>Dockerfile.snort3</i>)	N/A	N/A	<i>snort_data:/etc/snort</i>	snorty	pentest-network	Passive monitor, inspects network traffic, no exposed ports

All containers are connected via an internal Docker bridge network (pentest-network) and some services expose ports to the host system for GUI or web access. The systems are also installed with the necessary tools for security experiments as summarised in Table 1.

Networking and ports summary

All containers are interconnected via a Docker-defined bridge network called pentest-network. This simulates a LAN environment, enabling lateral movement and realistic attacker-defender interaction. Only selected ports are exposed to the host for access:

- DVWA: 8080:80 (Web UI)
- Kali: 7002:6901 (VNC GUI)
- Metasploitable: 7003:80 (Web UI)
- Snort3: Internal monitoring only (no exposed ports)

This setup creates a sandboxed penetration testing environment ideal for training, red/blue team practice, IDS evaluation, and exploit testing.

Understanding the *compose.yml* file: A closer look at Docker networking and setup

The *compose.yml* file you've provided is a Docker Compose configuration that

sets up a multi-container security test bed using three essential services: DVWA (Damn Vulnerable Web Application), Kali Linux, and Metasploitable2. Each of these containers has specific roles in the security lab environment, and this setup is orchestrated through Docker Compose to ensure that all services can interact seamlessly. The file also includes a custom network configuration that ties these containers together.

Let's break down each section of the *compose.yml* file, with a focus on Docker networking and how it enables communication between containers in this post-installation environment.

- *Version declaration* - '3': At the top of the file, we declare the version of the Docker Compose syntax being used.
- *Version* - '3': Version 3 is the most widely used version of Docker Compose, supporting advanced features like multi-host networking, service scaling, and load balancing. It ensures compatibility across different Docker versions and environments.

Services section: The core of the *compose.yml* file is the services section, where we define the three containers

that will form the security test bed. Each service corresponds to a container and has its own configuration settings like ports, environment variables, and networking as summarised in Table 1.

Networks section: At the bottom of the *compose.yml* file, we define the custom network that all three services (containers) will use:

```
networks:
  pentest-network:
    driver: bridge
```

The pentest-network is a Docker network using the bridge driver. In Docker, networks enable containers to communicate with each other and the host machine. The bridge driver is the default network driver, which provides isolated networking between containers on the same host while still allowing external access via port mappings.

How Docker networking works

Containers on the same bridge network can communicate with each other using their container names (e.g., *dvwa*, *kali*, *snort3*, *metasploitable*). External access to services is controlled by port mappings. For example, port 7001